DBMS

MGU BCA - 3RD SEM

Database Management Systems

Complete Important Questions

By Aben Sojan

Exam - 6 Nov 2026

MODULE 1

Introduction to Databases

1

EXAM PATTERN (from real MGU question papers)

MGU BCA Honours DBMS (MG3CCRBCA201): 70 marks, 2 hours. Part A: 5 x 2 marks, short questions from all modules. Part B: answer any 5 of 6-mark questions. Part C: answer any 2 of 15-mark essays. Old CBCS papers (2019-2021): 80 marks, 3 hours, any-10 x 2 + any-6 x 5 + any-2 x 15. Part C hotspots: COMPANY ER diagram, Normal Forms essay, anomalies.

Every module, every important, repeated and sure-mark question for Database Management Systems, each with a full-mark model answer.

1 Define data, database and DBMS.

IMPORTANT

Data means raw, unprocessed facts and figures that have no meaning by themselves, like a name, a mark or a price.

For example, the values 'Aben', 85 and 'Kottayam' are data, but they tell us nothing until they are organised.

A database is an organised collection of related data stored together so that it can be used easily.

For example, all the student records of a college, with names, roll numbers and marks, form a student database.

A DBMS (Database Management System) is software that creates, stores, updates and retrieves data from a database.

It acts as an interface between the users and the database, so users do not need to know how data is stored on disk.

A DBMS also manages security, backup and sharing of data among many users at the same time.

Examples of DBMS software are MySQL, Oracle, PostgreSQL and MongoDB.

2 What is a DBMS? Give two examples.

IMPORTANT

A Database Management System (DBMS) is a software system that enables users to create, store, retrieve, update and manage data in databases in an organised and efficient way.

It works as a bridge between the users and the physical database, so users can ask for data in a simple language like SQL without knowing how it is stored.

A DBMS allows many users to access the same data at the same time without conflicts, using locking and concurrency control.

It provides security through user accounts, passwords and access permissions, so only authorised users can see or change sensitive data.

It keeps data consistent and correct using constraints like primary key and foreign key, which block invalid entries.

It includes backup and recovery tools, so data can be restored if the system crashes or a disk fails.

Two popular examples of DBMS are MySQL, which is widely used in web applications, and Oracle, which is used in large enterprise systems.

Other well-known examples include PostgreSQL and MongoDB.

3 List any four applications of databases with one line each.

IMPORTANT

Banking: a DBMS stores customer account details and records every deposit, withdrawal and ATM transaction safely.

Education: colleges use a DBMS to manage student admissions, marks, attendance and fees in one place.

Railway reservation: a DBMS books tickets and tracks seat availability live for thousands of passengers at once.

E-commerce: websites like Flipkart and Amazon use a DBMS to manage products, orders, payments and customer details.

Hospitals: a DBMS stores patient records, doctor details and appointment schedules so that any doctor can see a patient's history quickly.

Libraries: a DBMS tracks books, members and issue and return dates, making searching and lending simple.

These applications are possible because a DBMS handles large amounts of shared data quickly and reliably.

4 Why do banks and railways prefer DBMS over plain files?

IMPORTANT

Banks and railways prefer a DBMS over plain files because thousands of users need to access and update the same data at the same time safely.

A DBMS supports concurrent access with locking, so two clerks booking tickets or updating accounts never overwrite each other's work.

Every transaction is recorded completely through the ACID properties, so there are no half-written updates like money debited but not credited.

Data stays consistent everywhere: once a ticket is booked, it instantly becomes unavailable to every other user.

A DBMS enforces rules and access control, so only authorised staff can see or change sensitive records like account balances.

Regular backups and recovery tools protect years of records from being lost in a crash.

Plain files cannot offer any of this, so they become unmanageable and unsafe at the scale of banks and railways.

5 Explain the advantages of DBMS over the traditional file system. (Oct 2019, Dec 2021)

REPEATED

The traditional file system stores data in separate files for each application, while a DBMS stores it once in a central shared database, which gives many advantages.

Less redundancy: the same data is not repeated in many files, so storage is not wasted and there is only one copy to maintain.

Data consistency: because there is one shared copy, an update made once is reflected everywhere, so copies never mismatch.

Data sharing: many users and programs can use the same data at the same time through controlled concurrent access.

Data security: user accounts, passwords and access permissions protect sensitive data from unauthorised users.

Data integrity: rules like primary key, foreign key and CHECK constraints keep data correct, for example marks always stay between 0 and 100.

Backup and recovery: the DBMS can take automatic backups and restore data after crashes, hardware failures or power cuts.

Easy querying: languages like SQL let users get answers quickly without writing complex file-handling programs.

6 What is data redundancy? How does DBMS reduce it?

IMPORTANT

Data redundancy means the same data is stored again and again in different files, for example a student's address repeated in the admission file, the library file and the hostel file.

It wastes storage space and causes inconsistency, because if one copy is updated and another is not, the copies disagree with each other.

A DBMS reduces redundancy by storing data once in a central database that every application and user shares.

Instead of repeating the address in every department's file, it is stored once in a Student table and all departments read it from there.

When the address changes, it is updated in one place and the change is instantly visible to everyone.

This is why redundancy in a DBMS is controlled and minimal, unlike in the traditional file system where duplication grows out of control.

7 What are the disadvantages of DBMS?

IMPORTANT

High cost: DBMS software licences, powerful hardware and training for staff make it expensive to set up and run.

Large and complex: a DBMS needs much more memory, disk space and processing power than simple files, and it is harder to manage.

Needs trained staff: a skilled DBA (Database Administrator) is required to install, secure, tune and maintain the system.

Single point of failure: if the central DBMS goes down, every application that depends on it stops working at the same time.

Performance overhead: features like locking, logging and security checks add extra work, which can slow down very simple tasks.

Conversion effort: moving old file-based data into a DBMS takes time, planning and careful data cleaning.

8 DBMS has disadvantages, yet organisations still use it. Why?

IMPORTANT

Organisations still use a DBMS because its benefits, like consistency, security, sharing and integrity, far outweigh its costs.

For large shared data, plain files become unmanageable very quickly: copies multiply, updates clash and searching becomes painful.

A DBMS solves these problems in one place, so the business can trust its data even with thousands of daily transactions.

Modern hardware and cloud options have reduced the cost problem, since a DBMS can now be rented cheaply instead of bought.

The cost of data loss or wrong data, like a wrong bank balance, is far higher than the cost of the DBMS itself.

So almost every serious application today, from banking to e-commerce, runs on a DBMS.

9 Explain the different types of database users. (Mar 2021)

IMPORTANT

Naive users are end users who interact with the database through ready-made application screens without knowing SQL, for example a bank clerk or a railway booking clerk.

Application programmers are software developers who write programs that access the database, for example the team that builds a banking app.

Sophisticated users are skilled people like data analysts who write their own SQL queries to get specific answers from the database.

They understand the database structure well enough to ask ad-hoc questions without a programmer's help.

Specialised users are another category in some books, using custom database applications like a graphics designer using a design database.

The DBA (Database Administrator) is the most privileged user, who administers, secures and maintains the entire database system.

Together these users cover everyone from the casual clerk to the expert who keeps the whole system running.

10 What are the roles and responsibilities of a DBA? (Mar 2021)

IMPORTANT

The DBA (Database Administrator) is the person responsible for managing the entire database system.

The DBA designs the database schema, deciding the structure of tables, keys and relationships.

The DBA creates user accounts and grants access rights, deciding who can read or modify which data.

The DBA takes regular backups and performs recovery after crashes, so no data is ever lost.

The DBA monitors performance and tunes the database, for example by creating indexes, so queries run fast.

The DBA enforces integrity and security rules, making sure invalid data cannot enter the database.

The DBA also manages storage space and plans for growth as data increases.

In short, the DBA is the caretaker of the database, keeping it correct, secure, fast and always available.

11 Who is a database designer? (Dec 2021)

IMPORTANT

A database designer is the person who plans the structure of the database before it is built.

The designer studies what data the organisation needs and how it will be used.

Then the designer identifies the entities (like Student and Course), their attributes, and the relationships between them, usually with an ER diagram.

The designer chooses the keys, removes redundancy through normalization, and writes the table schemas.

In short, the designer decides WHAT the database will store and HOW it will be organised, while the DBA later manages its daily running.

Good design done once saves years of pain: most anomalies and performance problems trace back to design mistakes.

12 Explain the different data models with one example each. (Dec 2021)

IMPORTANT

A data model is a way of structuring and organising data in a database, deciding how data is stored, linked and queried.

Hierarchical model: data is arranged in a tree structure with parent-child links, where each child has exactly one parent, for example a company organisation chart with managers and employees.

Network model: data is arranged in a graph structure where a child record can have many parents, allowing many-to-many links, for example a student linked to many courses and each course linked to many students.

Relational model: data is stored in tables with rows and columns linked through keys, for example a Student table in MySQL with columns rollno, name and marks.

Object-oriented model: data is stored as objects that bundle attributes with methods, for example a Student object holding name and marks along with a calculateGrade method.

The relational model is the most widely used today, while the object-oriented model suits complex applications like CAD systems.

13 Why are data models important in DBMS?

IMPORTANT

Data models are important because they decide how data is organised and stored inside the database.

They define how records or tables are linked to each other, such as a student linked to the courses he enrolled in.

They decide how users query and update the data, since each model has its own style of asking questions.

A good data model keeps data consistent, avoids duplication and makes searching fast and simple.

A poor choice of model makes the system slow, hard to change and full of errors.

For example, a billing system fits the relational model, while a social network's friend connections fit a graph-style model.

So choosing the right data model is the first and most important step in database design.

14 Define super key, candidate key and primary key with an example.

IMPORTANT

A super key is any set of columns that can uniquely identify each row in a table, for example {rollno, name} or even {rollno, name, marks}.

A candidate key is a minimal super key, meaning no column can be removed from it without losing uniqueness, for example rollno alone or email alone.

A primary key is the candidate key chosen by the designer to actually identify rows in the table, for example rollno.

The primary key has two strict rules: it can never be null and it can never be repeated in two rows.

Consider a Student table with columns rollno, name and email. Here {rollno, name} is a super key, rollno and email are candidate keys, and the designer may choose rollno as the primary key.

Candidate keys that are not chosen become alternate keys.

15 What is a foreign key? Explain with an example. (Model paper)

IMPORTANT

A foreign key is a column in one table that refers to the primary key of another table, creating a link between the two tables.

It guarantees referential integrity, meaning every value in the foreign key must exist as a primary key value in the parent table.

For example, consider Student(rollno, name) where rollno is the primary key, and Marks(rollno, subject, mark).

In the Marks table, rollno is a foreign key that refers to Student(rollno), linking each mark row to the correct student.

Because of the foreign key, nobody can insert marks for a roll number that does not exist in the Student table.

Foreign keys are the main tool for connecting related tables in the relational model.

16 Distinguish between primary key and unique key.

IMPORTANT

A primary key uniquely identifies each row, and there can be only one primary key per table.

The primary key never allows null values and never allows duplicate values.

A unique key also bans duplicate values, but a table can have many unique keys.

The unique key allows one null value, since null is not treated as a duplicate.

For example, in a Student table, rollno can be the primary key, while email can be a unique key.

This means two students cannot share the same email, but a student whose email is not known can have a null there.

In short, use the primary key for the main row identifier and unique keys for other columns that must stay distinct.

17 What are composite and surrogate keys? Give examples.

IMPORTANT

A composite key is a primary key made of two or more columns taken together, used when no single column is unique.

For example, in a Marks table, (rollno, subject) can be the composite primary key, because one student can have many subjects and one subject can belong to many students, but the pair is unique.

A surrogate key is an artificial key added by the system that has no real-world meaning, for example an auto-increment id column.

Surrogate keys are used when no natural column is a good unique identifier, or when natural keys are long and awkward.

For example, a Customer table may use an auto-increment customer_id even though phone numbers exist, because phone numbers can change.

Composite keys come from the data itself, while surrogate keys are invented by the database for convenience.

18 Can a primary key be NULL or duplicate? Justify. (Model paper)

IMPORTANT

No. A primary key can be neither NULL nor duplicate; both are forbidden by definition.

The primary key uniquely identifies each row in a table, so every row must have exactly one distinct, non-empty value.

NULL means unknown or missing, and an unknown value cannot identify a row, so the DBMS rejects NULL in a primary key column.

Duplicates are rejected because two rows with the same key value would be indistinguishable, which breaks the whole purpose of the key.

For example, in Student(rollno, name) with rollno as primary key, two students cannot share rollno 101, and rollno cannot be left blank.

If you try an INSERT with a duplicate or NULL primary key, the DBMS raises a constraint-violation error and the row is not added.

This rule is called entity integrity: it guarantees every row is uniquely and permanently identifiable.

In short: no NULLs, no duplicates - that is what makes a primary key reliable.

19 Explain the different table constraints in DBMS. (Model paper)

IMPORTANT

Table constraints are rules enforced by the DBMS on the data stored in a table, keeping every entry valid.

PRIMARY KEY: uniquely identifies each row; the column cannot be null and cannot have duplicates.

FOREIGN KEY: ensures the link between two tables stays valid by allowing only values that exist as a primary key in the parent table.

UNIQUE: duplicate values are not allowed in the column, though one null is permitted.

NOT NULL: the column must always have a value and cannot be left empty.

CHECK: only values satisfying a given condition are allowed, for example marks between 0 and 100.

DEFAULT: fills in a fixed value automatically when no value is given for the column.

Together these constraints protect the database from wrong, missing or inconsistent data.

20 What is the CHECK constraint? Give an example with SQL.

IMPORTANT

The CHECK constraint allows only those values in a column that satisfy a given condition, rejecting everything else.

It is used to enforce business rules directly in the database, so no program can store invalid data by mistake.

For example, marks in a Student table must always be between 0 and 100.

This rule can be written in SQL while creating the table:

```
CREATE TABLE Student (
rollno INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
marks INT CHECK (marks >= 0 AND marks <= 100)
);
```

If anyone tries to insert marks like 150 or -10, the DBMS rejects the row with an error.

This keeps the data correct without every application having to check the rule itself.

21 What is referential integrity constraint? (Oct 2019)

IMPORTANT

Referential integrity is the rule that a foreign key value must always match an existing primary key value in the referenced table, or be NULL.

It keeps relationships between tables valid: you cannot point to a row that does not exist.

For example, in Enroll(stud_id, course_id) where stud_id references Student(rollno), every stud_id in Enroll must exist as a rollno in Student.

If you try to insert an enrollment for rollno 999 when no such student exists, the DBMS rejects it.

It also controls deletes and updates: deleting a student who still has enrollments is blocked (or cascaded), so no orphan rows are left behind.

The constraint is declared with FOREIGN KEY ... REFERENCES in the CREATE TABLE statement.

Without it, the database could fill up with dangling references pointing nowhere.

So referential integrity guarantees that links between tables always point to real, existing rows.

MODULE 2

ER and Relational Model, SQL

2

EXAM PATTERN (from real MGU question papers)

MGU BCA Honours DBMS (MG3CCRBCA201): 70 marks, 2 hours. Part A: 5 x 2 marks, short questions from all modules. Part B: answer any 5 of 6-mark questions. Part C: answer any 2 of 15-mark essays. Old CBCS papers (2019-2021): 80 marks, 3 hours, any-10 x 2 + any-6 x 5 + any-2 x 15. Part C hotspots: COMPANY ER diagram, Normal Forms essay, anomalies.

1 What is an ER model? Explain entities, attributes and relationships. (Dec 2021)

IMPORTANT

The ER (Entity-Relationship) model is a high-level conceptual design tool used to describe the data needed by an organisation and how that data is connected, before any tables are created.

An entity is a real-world object that we want to store data about, such as a student, a course or an employee.

An entity set is a collection of similar entities, for example the set of all students in a college.

An attribute is a property that describes an entity, such as name, roll number or marks of a student.

Each attribute has a value for every entity; the values taken together describe that entity completely.

A relationship is an association between two or more entities, for example a student 'enrolls in' a course.

Relationships have degree: a relationship between two entity sets is binary, between three is ternary.

Relationships can also carry their own attributes, called descriptive attributes, like the date of enrolment.

Example: Student (rollno, name) and Course (code, title) with the relationship Enrolls between them gives the design 'students enrol in courses'.

2 Explain the types of attributes with examples. (Dec 2021)

IMPORTANT

Attributes describe entities, and they are classified into types based on how their values behave.

A simple attribute cannot be divided further into meaningful parts, for example age or date of birth.

A composite attribute is made of smaller parts that are meaningful on their own, for example name split into firstname and lastname, or address split into city, street and pincode.

A single-valued attribute holds at most one value per entity, for example roll number.

A multi-valued attribute can hold many values for one entity, for example a student with two phone numbers; it is shown with a double ellipse in an ER diagram.

A derived attribute is calculated from another stored attribute instead of being stored directly, for example age derived from date of birth, or total marks derived from subject marks.

A key attribute uniquely identifies each entity in a set, for example rollno for students; it is shown with an underline.

A null-valued attribute may be missing for some entities, for example email, when a student has not given one.

Example summary: a Student entity can have simple name parts, multi-valued phone numbers, derived age, and key rollno.

3 What is an entity in the ER model? (Oct 2019)

IMPORTANT

An entity is a real-world object or concept about which data is stored, like a student, a book or a department.

Each entity has attributes: properties describing it. For example, a student entity has rollno, name and marks.

An entity type (like STUDENT) is the category; an entity instance is one member, e.g. the student Aben with rollno 101.

A group of similar entities forms an entity set, e.g. all students of the college.

In an ER diagram an entity type is drawn as a rectangle, with its key attribute underlined.

Getting entities right is the first step of database design - everything else (relationships, tables, keys) builds on it.

4 Compare entity type and entity set. (Mar 2021)

IMPORTANT

Entity type is the schema-level definition: the name plus its attributes, e.g. STUDENT(rollno, name, marks).

Entity set is the actual collection of entity instances of that type at a moment, e.g. the 60 student rows currently stored.

Type is like a class in programming; the set is like the objects created from it.

The type rarely changes, while the set changes every time a row is inserted or deleted.

Example: EMPLOYEE is the entity type; the 200 employee records in the table form the entity set.

Examiners love this distinction because confusing the schema with the data is the most common beginner mistake.

5 Explain relationship attributes with an example. (Mar 2021)

IMPORTANT

A relationship attribute is a property that belongs to the relationship itself, not to either participating entity.

It is needed when the fact only makes sense for the combination. Example: Hours in WORKS_ON.

Employee E1 works on project P1 for 10 hours and on P2 for 20 hours - hours cannot sit on EMPLOYEE or PROJECT alone; it belongs to the (E1, P1) pair.

Another example: StartDate on MANAGES - the date a manager took charge of a department describes the manages-relationship.

In ER diagrams, relationship attributes are drawn as ovals attached to the diamond.

Spotting relationship attributes correctly is what separates a correct ER diagram from a wrong one.

6 **When we think of relationships as attributes, what are the value sets of these attributes? What class of data model is based on this concept? (Oct 2019)**

IMPORTANT

If a relationship is viewed as an attribute of an entity, its value is a set of references to the related entities.

Example: the works-for relationship seen from EMPLOYEE is an attribute whose value is the department entity it points to.

So the value set of such an attribute is a set of entity references (object identifiers), not simple values.

The class of data model based on this idea is the object-oriented / object data model (and EER), where relationships are stored as reference-valued attributes.

This is why object databases navigate via stored references instead of joins.

This question tests whether you understand the idea behind object-oriented databases, not just ER notation.

7 **Construct an ER diagram for the COMPANY database with departments, projects, employees and dependants. (Oct 2019, Mar 2021)**

SURE MARK

Step 1 - entity types: DEPARTMENT, PROJECT, EMPLOYEE, and DEPENDENT (a weak entity).

DEPARTMENT attributes: Dname, Dnumber (key), locations (multi-valued, shown with a double oval).

PROJECT attributes: Pname, Pnumber (key), Plocation; each project is controlled by one department.

EMPLOYEE attributes: Fname, Minit, Lname, Ssn (key), Address, Salary, Sex, Bdate.

Step 2 - relationships: MANAGES is 1:1 between DEPARTMENT and EMPLOYEE, with attribute Mgr_start_date.

CONTROLS is 1:N from DEPARTMENT to PROJECT; WORKS_FOR is N:1, every employee works for exactly one department (total participation on the employee side).

WORKS_ON is M:N between EMPLOYEE and PROJECT with the relationship attribute Hours (hours per week).

SUPERVISION is a 1:N recursive relationship: one employee supervises many, each has at most one direct supervisor.

DEPENDENT is weak: attributes Dependent_name (partial key, dashed underline), Sex, Bdate,

Relationship; it depends on EMPLOYEE through the identifying relationship DEPENDENTS_OF (double diamond, double rectangle, total participation).

Draw rectangles for entities, diamonds for relationships, ovals for attributes; underline key attributes.

This exact question repeated verbatim in the Oct 2019 and Mar 2021 Part C - learn the spec cold.

8 Explain an ER diagram with the symbols used. (Model paper)

REPEATED

An ER diagram is a pictorial representation of the ER model, drawn with fixed symbols so that the database design can be seen at a glance.

A rectangle represents an entity set, for example a rectangle labelled Student.

An ellipse represents an attribute, for example an ellipse labelled name attached to the Student rectangle.

An underlined attribute (or underlined ellipse) is a key attribute, for example rollno underlined under Student.

A diamond represents a relationship set between entities, for example a diamond labelled Enrolls between Student and Course.

Straight lines connect an entity set to its relationship diamond and to its attribute ellipses.

A double ellipse marks a multi-valued attribute, and a double rectangle marks a weak entity set.

A double diamond marks an identifying relationship used by a weak entity set.

Cardinality constraints are written near the lines, such as 1 or N, showing how many entities of one side connect to the other.

Example: draw Student (rectangle) with name (ellipse) and rollno (underlined), Course (rectangle) with code (underlined), and connect both through the Enrolls (diamond).

9 What is a weak entity set? Give an example.

IMPORTANT

A weak entity set is an entity set that does not have a key attribute of its own, so its entities cannot be identified without help.

It depends on a strong entity set called the owner (or identifying) entity through a relationship called the identifying relationship.

A weak entity is identified by the key of its owner plus its own partial key (discriminator), which is only unique within one owner.

Example: Dependent (name, age) is weak because two employees can both have dependents named Raju; only Employee ID plus name makes a dependent unique.

In an ER diagram a weak entity set is drawn with a double rectangle, and its identifying relationship with a double diamond.

The discriminator attribute is drawn with a dashed underline.

The relationship from a weak entity to its owner is always many-to-one and mandatory: every weak entity must be linked to an owner.

Weak entities are converted to tables later by adding the owner's primary key to them, making a combined key.

10 Explain cardinality and participation constraints in ER diagrams. (Dec 2021)

IMPORTANT

Cardinality ratio says how many entities on one side can relate to how many on the other: 1:1, 1:N or M:N.

Example: DEPARTMENT-EMPLOYEE works-for is 1:N (one department, many employees); STUDENT-COURSE enrolls is M:N.

Participation says whether every entity must take part: total (double line) or partial (single line).

Example: every EMPLOYEE must work for a DEPARTMENT (total participation of EMPLOYEE), but a DEPARTMENT may have zero projects (partial).

Together they are called structural constraints, and they decide the final table design and foreign keys.

These two constraints together are called structural constraints, and they directly decide your foreign keys.

11 Explain the relational model with an example.

IMPORTANT

The relational model stores data in two-dimensional tables called relations, where each table represents one entity set or one relationship.

A relation has a fixed set of columns called attributes, and each column has a name and a data type.

The rows of a table are called tuples, and each tuple stores the data of one entity, like one student.

Values must be atomic (indivisible), so no cell holds a list of values.

Each relation has a primary key: an attribute or set of attributes that is unique and never null, such as rollno for Student.

A foreign key is an attribute in one table that refers to the primary key of another table, linking the two tables.

Example: Student(rollno, name, marks) with rollno as primary key; Enrolment(rollno, code) with rollno as a foreign key pointing to Student.

A relational schema is the design of these tables: the table names, their attributes, the keys and the foreign key links.

The relational model is simple, flexible and is the basis of almost every commercial DBMS like MySQL and Oracle.

12 What are degree and cardinality of a relation?

IMPORTANT

Degree of a relation is the number of attributes (columns) in the relation.

Cardinality of a relation is the number of tuples (rows) currently stored in it.

Degree is part of the schema and stays fixed, while cardinality keeps changing as rows are inserted or deleted.

Example: Student(rollno, name, marks) has degree 3 because it has three attributes.

If the college admits 60 students, the cardinality of that relation is 60.

When one new student joins, the cardinality becomes 61 but the degree remains 3.

Both terms describe the shape of the relation: degree is its width and cardinality is its height.

13 Compare relation, relationship type, relationship set and structural constraints. (Dec 2021)

IMPORTANT

Relation: a table in the relational model - rows (tuples) and columns (attributes), e.g. the STUDENT table.

Relationship type: an ER-level association between entity types, e.g. WORKS_FOR between EMPLOYEE and DEPARTMENT.

Relationship set: the actual pairs linked right now, e.g. the 200 (employee, department) pairs stored.

Structural constraints: the cardinality plus participation rules on a relationship type, e.g. 1:N with total participation.

In short: relationship type and set live in the ER design; the relation is the table you finally create; structural constraints are the rules shaping both.

This was a 15-mark Part C question - the examiner expects all four terms defined and contrasted, not just listed.

14 Define domains and tuples. (Dec 2021)

IMPORTANT

A domain is the set of allowed values for an attribute. For example, the domain of Age is integers from 0 to 150.

It works like a data type plus a permitted range, and it keeps invalid values out.

A tuple is one row of a relation: an ordered set of values, one from each attribute's domain.

Example: (101, 'Aben', 20, 85) is a tuple of STUDENT; the 20 comes from the Age domain.

So domains define what values are legal; tuples are the actual stored rows built from them.

Domains and tuples are the two building blocks of the relational model - every table is a set of tuples over domains.

15 What are the characteristics of relations? (Mar 2021)

IMPORTANT

Each relation (table) has a unique name, different from every other table.

Each attribute (column) has a unique name within the table.

All values in one column come from the same domain and are atomic (single-valued).

Each tuple (row) is unique - no two rows are fully identical.

The order of rows and of columns does not matter; the table means the same rearranged.

These rules are what make a relation stricter than an ordinary file or spreadsheet table.

16 Explain selection, projection and join in relational algebra.

IMPORTANT

Relational algebra is a procedural query language: queries are written as step-by-step operations on relations, each producing a new relation.

Selection picks the tuples (rows) that satisfy a given condition and keeps all attributes.

It is written with the sigma symbol, for example sigma with condition marks > 80 applied to Student returns only the students scoring above 80.

Projection picks the attributes (columns) wanted and drops the rest, automatically removing duplicate rows.

It is written with the pi symbol, for example pi with attribute name applied to Student returns just the list of student names.

Join combines tuples from two relations using a common attribute, for example Student joined with Marks on rollno.

The most common join is the natural join, which matches rows automatically on same-named columns.

Join is used whenever data needed for an answer is split across two tables, like names in Student and marks in Marks.

Example idea: to get the names of students scoring above 80, first select marks > 80 from Marks, then join with Student, then project name.

17 What are the set operations in relational algebra?

IMPORTANT

Relational algebra supports set operations that treat relations like mathematical sets of tuples.

Union combines all tuples appearing in either of the two relations, removing duplicates.

Intersection keeps only the tuples that appear in both relations.

Difference keeps the tuples that are in the first relation but not in the second.

Set operations can only be applied to union-compatible relations: both must have the same number of attributes with matching data types.

Example: if Pass has the roll numbers of students who passed and Distinction has the roll numbers with distinction, Pass UNION Distinction gives every passing roll number once.

Pass INTERSECT Distinction gives the distinction holders, and Pass MINUS Distinction gives those who passed without distinction.

Union is useful for combining results, intersection for common data, and difference for 'in A but not in B' questions.

18 Distinguish between tuple and domain relational calculus.

IMPORTANT

Relational calculus is a non-procedural query language: you describe what result you want, and the DBMS decides how to fetch it.

In tuple relational calculus, the variables range over whole tuples (rows) of a relation.

A typical tuple calculus query says 'find a tuple t such that t belongs to Student and t.marks > 80'.

In domain relational calculus, the variables range over individual attribute values (domains), not whole rows.

A typical domain calculus query says 'find a name value n such that there is a Student row with name n and marks > 80'.

Tuple calculus is closer to SQL row-wise thinking, while domain calculus matches SQL's column-wise SELECT list more directly.

Both are declarative and equally expressive: anything written in one can be written in the other.

Example idea: the question 'names of students with marks above 80' is expressed in both as a condition on the Student relation, with no steps mentioned.

19 Why is relational calculus called non-procedural?

IMPORTANT

Relational calculus is called non-procedural because the query only declares the desired result, never the steps to obtain it.

You state the condition the answer must satisfy, like 'marks greater than 80', without saying how to scan or index the table.

The DBMS query optimiser then works out the fastest execution plan on its own.

Relational algebra is the opposite: it is procedural, giving the exact sequence of selection, projection and join operations.

SQL follows the calculus idea: you write SELECT name FROM Student WHERE marks > 80 and let the engine plan the execution.

This makes queries shorter, easier to write and easier for the DBMS to optimise.

So 'non-procedural' simply means 'say what you want, not how to get it'.

20 Distinguish between DDL and DML with examples.

IMPORTANT

SQL commands are grouped by purpose: DDL defines and changes the structure of the database, while DML works with the data stored inside it.

DDL (Data Definition Language) includes CREATE TABLE, ALTER TABLE, DROP TABLE and TRUNCATE TABLE.

DDL commands change the schema itself, like adding a table or a column, and they usually commit automatically.

DML (Data Manipulation Language) includes INSERT, UPDATE, DELETE and SELECT.

DML commands change or read the rows of a table, and their changes can be rolled back with ROLLBACK before they are committed.

Example DDL: CREATE TABLE Student (rollno INT PRIMARY KEY, name VARCHAR(50)); creates the structure of the table.

Example DML: INSERT INTO Student VALUES (1, 'Aben', 85); adds one row of data to that table.

In short: DDL builds and modifies the container, DML fills, updates and reads what is inside it.

21 Write DDL commands to create a Student table and then add a column to it.

IMPORTANT

DDL is used to define the structure of the table, starting with the CREATE TABLE command.

The Student table is created with rollno as an integer primary key, name as a required text field, and marks as an integer.

The PRIMARY KEY constraint keeps rollno unique and non-null, and NOT NULL forces every row to have a name.

Later, when an email address is needed, ALTER TABLE adds the new column without touching the existing rows.

Code lines (unchanged):

Create the table with a primary key:

```
CREATE TABLE Student (
rollno INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
marks INT
);
```

Add a new column later:

```
ALTER TABLE Student ADD email VARCHAR(60);
```

Explanation: CREATE TABLE defines the new relation and its constraints in one step.

ALTER TABLE with ADD appends the email column to the existing table, and old rows simply get a null email until updated.

22 What is SQL? (Oct 2019)

IMPORTANT

SQL (Structured Query Language) is the standard language for working with relational databases.

It is used to create tables, insert and update data, and ask questions (queries) of the database.

It has sub-languages: DDL (CREATE, ALTER, DROP), DML (SELECT, INSERT, UPDATE, DELETE), DCL and TCL.

Example: SELECT name FROM Student WHERE marks > 80; lists the names of high scorers.

Almost every relational DBMS - MySQL, Oracle, PostgreSQL - understands SQL, so the skill transfers between systems.

SQL is the one language every database job expects, which is why it opens the practical module too.

23 What are the basic DDL commands in SQL? (Dec 2021)

IMPORTANT

DDL (Data Definition Language) commands define or change the structure of tables.

CREATE makes a new table:

```
CREATE TABLE Student(rollno INT PRIMARY KEY, name VARCHAR(30));
```

ALTER changes an existing table:

```
ALTER TABLE Student ADD COLUMN age INT;
```

DROP deletes a table completely:

```
DROP TABLE Student;
```

TRUNCATE removes all rows but keeps the structure:

```
TRUNCATE TABLE Student;
```

DDL commands take effect immediately and cannot be rolled back in most systems.

24 Write the syntax of the SELECT command with an example. (Dec 2021)

IMPORTANT

Syntax:

```
SELECT column1, column2 FROM table_name WHERE condition;
```

SELECT * picks all columns; WHERE filters rows.

Example:

```
SELECT name, marks FROM Student WHERE marks > 80;
```

This returns only the name and marks columns of students scoring above 80.

Example with all columns:

```
SELECT * FROM Student;
```

This lists the full table.

25 Give the general form of a SQL query. (Mar 2021)

IMPORTANT

General form:

```
SELECT <attribute list>
FROM <table list>
WHERE <condition>
GROUP BY <grouping attributes>
HAVING <group condition>
ORDER BY <attribute list>;
```

SELECT and FROM are compulsory; the rest are optional.

The engine filters rows (WHERE), groups them (GROUP BY), filters groups (HAVING), then sorts (ORDER BY).

Example:

```
SELECT dept, AVG(marks) FROM Student GROUP BY dept HAVING AVG(marks) > 70;
```

26 **Describe the four clauses in the syntax of a simple SQL retrieval query. (Oct 2019)**

IMPORTANT

SELECT: lists which columns to show, e.g. SELECT name, marks.

FROM: names the table (or tables) to read, e.g. FROM Student.

WHERE: filters individual rows before any grouping, e.g. WHERE marks > 80.

ORDER BY: sorts the final result, e.g. ORDER BY marks DESC.

Together:

```
SELECT name, marks FROM Student WHERE marks > 80 ORDER BY marks DESC;
```

This shows high scorers with the best first.

27 **Consider the table Student(RollNo, Name, Age, Marks). Write SQL queries to: (a) add a new column Percentage, (b) find students whose names start with 'A', (c) find students aged between 18 and 22, (d) find the minimum and maximum marks. (Model paper)**

IMPORTANT

(a) Add the column with ALTER TABLE:

```
ALTER TABLE Student ADD COLUMN Percentage FLOAT;
```

(b) Names starting with A, using LIKE:

```
SELECT * FROM Student WHERE Name LIKE 'A%';
```

LIKE with 'A%' matches names starting with A; % means any following characters.

(c) Age range with BETWEEN:

```
SELECT * FROM Student WHERE Age BETWEEN 18 AND 22;
```

BETWEEN checks an inclusive range, the same as Age >= 18 AND Age <= 22.

(d) Extremes with aggregate functions:

```
SELECT MIN(Marks), MAX(Marks) FROM Student;
```

MIN and MAX return the smallest and largest marks in the table.

28 Explain aggregate functions in SQL with examples. (Mar 2021)

IMPORTANT

Aggregate functions in SQL take many rows as input and return a single summary value.

COUNT(*) counts the number of rows in a table, including rows with nulls.

SUM(marks) adds up all the values of a numeric column, and AVG(marks) gives their average.

MIN(marks) returns the smallest value and MAX(marks) returns the largest value in the column.

Aggregates ignore null values except COUNT(*), so they work on the real data only.

Example query (unchanged):

```
SELECT COUNT(*), AVG(marks), MAX(marks) FROM Student;
```

Explanation: COUNT(*) tells how many students are in the table, AVG(marks) gives the class average, and MAX(marks) shows the top score.

The FROM clause picks the Student table, and all three aggregates are computed over its full set of rows in one pass.

Aggregates are the standard way to answer 'how many', 'average', 'total' and 'maximum' questions in exams.

29 Write a query to find the number of students with marks above 80.

IMPORTANT

To count only the students above a cut-off, COUNT(*) is combined with a WHERE condition.

The WHERE clause filters rows first, keeping only those with marks above 80.

COUNT(*) then counts exactly those filtered rows, so students with 80 or less are not counted.

Code line (unchanged):

Use COUNT with a WHERE condition:

```
SELECT COUNT(*) FROM Student WHERE marks > 80;
```

Explanation: FROM Student picks the table, WHERE marks > 80 keeps only the rows scoring above 80, and COUNT(*) returns how many such rows exist.

The condition uses the greater-than comparison, so a student with exactly 80 is excluded.

The result is a single number, for example 12, which is the count asked for.

30 Explain GROUP BY and HAVING with an example query. (Dec 2021, Oct 2019)

REPEATED

GROUP BY collects rows that share the same value in a column into separate groups, so aggregates can be computed per group.

HAVING filters these groups, in the same way that WHERE filters individual rows.

WHERE cannot be used with aggregate functions, but HAVING is designed exactly for conditions on aggregates.

The order of execution is: WHERE filters rows first, GROUP BY forms groups, HAVING filters the groups.

Example: to find departments with more than 5 students, rows are grouped by dept and groups with a small count are removed.

Code lines (unchanged):

GROUP BY collects rows with the same value into groups.

HAVING filters those groups, like WHERE filters rows.

Example: departments having more than 5 students:

```
SELECT dept, COUNT(*) FROM Student
GROUP BY dept HAVING COUNT(*) > 5;
```

Explanation: GROUP BY dept makes one group per department, COUNT(*) counts students in each group, and HAVING COUNT(*) > 5 keeps only the large departments.

The SELECT list shows the department name with its student count.

31 Distinguish between WHERE and HAVING.

IMPORTANT

WHERE and HAVING both filter data, but they work at different stages of a query.

WHERE filters individual rows before any grouping happens.

HAVING filters groups after GROUP BY has been applied.

WHERE cannot use aggregate functions like COUNT or AVG, because aggregates are computed after WHERE runs.

HAVING is meant for conditions on aggregates, like COUNT(*) > 5.

A query can use both: WHERE marks > 40 keeps only the passing rows, then GROUP BY forms groups, then HAVING COUNT(*) > 5 drops the small groups.

Example: WHERE marks > 40 picks the individual rows; HAVING COUNT(*) > 5 picks the qualifying groups.

So the rule is: filter rows with WHERE, filter groups with HAVING.

32 What is the use of LIKE and DISTINCT? Give examples.

IMPORTANT

LIKE is used in the WHERE clause to match text against a pattern.

The percent sign % stands for any number of characters, and the underscore _ stands for exactly one character.

So LIKE 'A%' matches every name starting with A, like Aben or Ansu.

Code line (unchanged):

```
SELECT * FROM Student WHERE name LIKE 'A%';
```

Explanation: WHERE name LIKE 'A%' keeps only the rows whose name begins with the letter A, and SELECT * returns all their columns.

DISTINCT removes duplicate values from the result, so each value appears only once.

Code line (unchanged):

```
SELECT DISTINCT dept FROM Student;
```

Explanation: without DISTINCT, one dept value would repeat for every student in it; DISTINCT collapses them to a single list of department names.

LIKE is for pattern searching and DISTINCT is for removing duplicates.

33 How can you order rows in SQL? (Dec 2021)

IMPORTANT

Rows are ordered with the ORDER BY clause at the end of a SELECT query.

Syntax:

```
SELECT columns FROM table ORDER BY column ASC | DESC;
```

ASC (ascending) is the default; DESC gives highest-first order.

Example:

```
SELECT name, marks FROM Student ORDER BY marks DESC;
```

This lists toppers first.

Multiple columns are allowed:

```
SELECT name, dept, marks FROM Student ORDER BY dept ASC, marks DESC;
```

This sorts by department, then by marks within each department.

34 Explain the different types of joins with examples. (Model paper)

IMPORTANT

A join combines rows from two tables using a related column, usually a primary key matching a foreign key.

INNER JOIN returns only the rows that have a match in both tables; unmatched rows from either side are dropped.

LEFT JOIN (left outer join) returns every row from the left table, filling right-side columns with null when there is no match.

RIGHT JOIN returns every row from the right table, filling left-side columns with null when there is no match.

FULL JOIN returns everything from both tables, with nulls filling whichever side has no match.

NATURAL JOIN joins automatically on all columns with the same name in both tables, like rollno.

Example with Student and Marks on rollno: INNER JOIN shows students who have marks rows; LEFT JOIN shows all students even those without marks.

The choice of join depends on the question: use INNER JOIN for 'only matching data' and LEFT JOIN for 'all from one side, data where available'.

35 Distinguish between INNER JOIN and LEFT OUTER JOIN.

IMPORTANT

INNER JOIN and LEFT OUTER JOIN differ in how they treat rows that have no match in the other table.

INNER JOIN returns only the rows where a matching row exists in both tables.

LEFT OUTER JOIN returns every row from the left table, whether or not a match exists.

For a left-table row with no match, the columns from the right table appear as null.

INNER JOIN therefore shrinks the result to matched pairs only, while LEFT JOIN keeps the full left side.

Example: joining Student with Marks, INNER JOIN hides students who have no marks rows, while LEFT JOIN still lists them with null marks.

So INNER JOIN answers 'which students have marks', and LEFT JOIN answers 'all students and their marks if any'.

36 Write a query using INNER JOIN to show student names with their marks.

IMPORTANT

To show each student's name together with their marks, the Student and Marks tables are joined on the common rollno column.

INNER JOIN keeps only the students who actually have marks rows, so the result has no unmatched entries.

The ON clause states the matching condition: Student.rollno = Marks.rollno.

Qualified column names like Student.name avoid confusion when both tables share column names.

Code lines (unchanged):

Join Student and Marks on the common rollno column:

```
SELECT Student.name, Marks.subject, Marks.mark
FROM Student INNER JOIN Marks
ON Student.rollno = Marks.rollno;
```

Explanation: FROM with INNER JOIN pairs each student row with its marks rows where the roll numbers are equal, and SELECT picks the name, subject and mark.

The ON clause performs the match, and the SELECT list decides which columns appear in the final answer.

37 Write a query using LEFT JOIN to list all students even if they have no marks.

IMPORTANT

LEFT JOIN is used when every student must appear in the result even if the Marks table has no row for them.

Unlike INNER JOIN, it never drops a left-table row: unmatched students are shown with null marks.

The ON clause still gives the matching condition Student.rollno = Marks.rollno.

This is the correct join for questions like 'list all students, with marks where available'.

Code lines (unchanged):

LEFT JOIN keeps every student row:

```
SELECT Student.name, Marks.mark
FROM Student LEFT JOIN Marks
ON Student.rollno = Marks.rollno;
```

Explanation: FROM Student LEFT JOIN Marks starts from all student rows and attaches marks rows where roll numbers match; missing marks show as null.

SELECT picks just the name and the mark, giving one row per student with a blank mark where no marks row exists.

38 What is the use of the INTERSECT operator? (Oct 2019)

IMPORTANT

INTERSECT returns only the rows common to two query results (set intersection).

Both queries must return the same number of columns with compatible types.

Example:

```
SELECT rollno FROM CricketTeam INTERSECT SELECT rollno FROM FootballTeam;
```

This gives the roll numbers of students who are in both teams.

Duplicates are removed automatically; only distinct common rows appear.

39 Explain UNIQUE and EXISTS in SQL. (Dec 2021)

IMPORTANT

UNIQUE is a constraint: it forbids duplicate values in a column but allows NULL.

Example:

```
CREATE TABLE Student(rollno INT PRIMARY KEY, email VARCHAR(50) UNIQUE);
```

It is weaker than PRIMARY KEY (which also forbids NULL) and suits alternate keys like email or phone.

EXISTS is a subquery test: WHERE EXISTS (subquery) is true when the subquery returns at least one row.

Example:

```
SELECT name FROM Student S WHERE EXISTS (SELECT * FROM Marks M WHERE M.rollno = S.rollno AND M.mark > 90);
```

So UNIQUE guards data quality at table-design time, while EXISTS filters rows at query time.

MODULE 3

Normalization and Transaction Management

3

EXAM PATTERN (from real MGU question papers)

MGU BCA Honours DBMS (MG3CCRBCA201): 70 marks, 2 hours. Part A: 5 x 2 marks, short questions from all modules. Part B: answer any 5 of 6-mark questions. Part C: answer any 2 of 15-mark essays. Old CBCS papers (2019-2021): 80 marks, 3 hours, any-10 x 2 + any-6 x 5 + any-2 x 15. Part C hotspots: COMPANY ER diagram, Normal Forms essay, anomalies.

1 What is modification anomaly? (Dec 2021, Mar 2021)

SURE MARK

Modification (update) anomaly: changing one fact forces changes in many rows, risking inconsistency.

It happens when the same fact is repeated because the table is not normalized.

Example: in Enroll(rollno, name, course), the student name Aben appears in 5 rows; correcting the spelling needs 5 updates.

If even one row is missed, the database holds two different names for the same student - inconsistent data.

Normalization removes the anomaly by storing each fact once: Student(rollno, name) plus Enroll(rollno, course).

This is why anomalies are the main motivation for normalization.

2 What is insertion anomaly? (Oct 2019)

IMPORTANT

Insertion anomaly: you cannot insert a fact because another unrelated fact is missing.

Example: in Dept_Emp(deptno, deptname, empno, empname), you cannot add a new department until it has at least one employee.

The department's existence wrongly depends on having employees.

After normalization into Department(deptno, deptname) and Employee(empno, empname, deptno), departments can be added freely.

This anomaly also wastes design freedom: the table structure forces a business rule that should not exist.

In short, insertion anomaly means missing data in one column blocks perfectly valid data in another.

3 What is deletion anomaly?

IMPORTANT

Deletion anomaly: deleting one fact accidentally deletes another unrelated fact.

Example: in Dept_Emp(deptno, deptname, empno, empname), deleting the last employee of a department wipes out the department's name too.

The department information is lost just because its employees left.

Normalization fixes it: deleting from Employee never touches the Department table.

It is the mirror of the insertion anomaly: two independent facts share one row, so removing one destroys the other.

This is why each entity type gets its own table during normalization.

4 What is a functional dependency? Explain its types. (Oct 2019)

IMPORTANT

A functional dependency X -> Y means that in a relation, each value of the attribute set X determines exactly one value of the attribute set Y.

It is a constraint on the data: if two rows agree on X, they must also agree on Y. Example: rollno -> name, because one roll number belongs to exactly one student.

FDs come in three types:

1. Trivial FD: Y is a subset of X, so it holds automatically. Example: {rollno, name} -> rollno.

2. Non-trivial FD: Y is not a subset of X, expressing a real rule of the data. Example: rollno -> name.

3. Transitive FD: X -> Y and Y -> Z hold, so X -> Z holds indirectly. Example: rollno -> dept and dept -> hod give rollno -> hod.

FDs are the raw material of normalization: an FD whose left side is not a key signals redundancy that a normal form will remove.

They also identify keys: if X determines every attribute of the relation, X is a super key.

5 **Give two examples of functional dependencies in a Student table.**

IMPORTANT

Example 1: rollno -> name. One roll number belongs to exactly one student, so knowing the roll number fixes the name.

Example 2: rollno -> dept. One roll number belongs to exactly one department, so knowing the roll number fixes the department.

Both are non-trivial FDs because the right side is not part of the left side; they capture real rules of the college data.

A non-example: name -> rollno is false, because two students can share the same name, so one name does not fix one roll number.

Another valid FD is {rollno, subject} -> marks: one student in one subject has exactly one mark.

Since rollno determines every column of the Student table, rollno is a super key, in fact the primary key.

In an exam, always test a claimed FD by asking: can one left-side value ever map to two different right-side values? If yes, it is not an FD.

These FDs are what normalization later uses to detect partial and transitive dependencies.

6 **Discuss the role of functional dependencies in determining keys, superkeys and candidate keys. (Model paper)**

IMPORTANT

A functional dependency X -> Y means X's value fixes Y's value; this is how keys are discovered.

A superkey is any attribute set whose closure includes all attributes of the table.

A candidate key is a minimal superkey - no smaller subset still determines everything.

Example: in Student(rollno, name, dept) with rollno -> name and rollno -> dept, the closure of {rollno} is all attributes, so rollno is a candidate key (and the primary key).

Armstrong's axioms and attribute closure are the mechanical tools used to test these.

So FDs are the foundation: keys are simply the minimal attribute sets that functionally determine the whole row.

7 State and explain Armstrong's axioms.

IMPORTANT

Armstrong's axioms are three inference rules that derive every FD logically implied by a given set of FDs.

1. Reflexivity: if Y is a subset of X, then X -> Y. Example: {rollno, name} -> rollno, since rollno is already inside the left side.

2. Augmentation: if X -> Y, then XZ -> YZ for any attribute set Z. Example: from rollno -> name we get {rollno, dept} -> {name, dept}.

3. Transitivity: if X -> Y and Y -> Z, then X -> Z. Example: rollno -> dept and dept -> hod give rollno -> hod.

The axioms are sound: they derive only FDs that truly hold. They are also complete: every FD that logically follows can be derived with them.

Useful derived rules include union (X -> Y and X -> Z give X -> YZ) and decomposition (X -> YZ gives X -> Y and X -> Z).

They are the engine behind closure computation: repeatedly applying the axioms to an attribute set yields everything it determines.

That is why closure, keys and normal-form tests all rest on these three simple rules.

8 Using Armstrong axioms, show that if A -> B and B -> C then A -> C.

IMPORTANT

Given: A -> B and B -> C hold in the relation.

The transitivity axiom states: if X -> Y and Y -> Z, then X -> Z.

Match X = A, Y = B, Z = C against the axiom's pattern.

Our two given FDs fit the pattern exactly: A -> B is X -> Y, and B -> C is Y -> Z.

Applying the axiom therefore yields A -> C.

This holds in every relation instance satisfying the given FDs, because the axiom is sound.

It shows why dependency chains collapse: a two-step determination is equivalent to a direct one.

Closure computation is exactly this proof repeated: from A we collect B, then from B we collect C.

9 What is attribute closure? How is it used?

IMPORTANT

The attribute closure of X, written X+, is the complete set of attributes functionally determined by X under the given FDs.

Computation method: start with the set containing X, then repeatedly find any FD whose left side is already inside the set and add its right side, stopping when nothing new can be added.

Example: with rollno -> dept and dept -> hod, the closure of {rollno} is {rollno, dept, hod}.

Use 1, finding keys: if X+ contains every attribute of the relation, then X is a super key.

Use 2, testing normal forms: we need X+ to know whether an attribute is prime (part of some key) and whether a FD's left side is a key.

Use 3, checking implication: the FD X -> Y is implied exactly when Y is inside X+.

Closure is therefore the working tool of normalization: it turns a small FD set into everything those FDs imply.

In the exam, always show each step of the closure build-up, since the working earns the marks.

10 Find the closure of {rollno} given rollno -> dept and dept -> hod.

IMPORTANT

Given FDs: rollno -> dept and dept -> hod. We build the closure of {rollno} step by step.

Step 1: start with the initial set {rollno}.

Step 2: the FD rollno -> dept has its left side inside the set, so add dept. The set becomes {rollno, dept}.

Step 3: the FD dept -> hod now applies because dept is in the set, so add hod. The set becomes {rollno, dept, hod}.

Step 4: check again; no FD can add anything new, so the process stops.

Therefore the closure of {rollno} is {rollno, dept, hod}.

This also proves the transitive FD rollno -> hod, since hod ended up inside the closure.

Note: if the relation is Student(rollno, name, dept, hod), this closure does not include name, so {rollno} alone is not a key of that full relation.

11 What is 1NF? Convert an unnormalised table to 1NF with an example. (Dec 2021)

IMPORTANT

First Normal Form requires that every cell of a table holds a single atomic value: no lists, no repeating groups, no multi-valued attributes packed into one column.

Example of violation: Student(rollno, name, phones) where one row stores phones = '9876543210, 9123456780' in a single cell.

This breaks queries: asking 'which student has phone 9123456780' needs string searching instead of a clean equality test.

Conversion to 1NF: split the multi-valued cell into separate rows, one value per row.

Result: (1, Aben, 9876543210) and (1, Aben, 9123456780), so each phone gets its own row and each cell holds exactly one value.

Every cell is now atomic, so the table satisfies 1NF.

The cost is some repeated data (rollno and name appear twice), which higher normal forms handle with keys.

1NF is the entry ticket: a relation must be in 1NF before 2NF, 3NF or BCNF can even be checked.

12 A table stores many hobbies in one column. Why does it violate 1NF? Fix it.

IMPORTANT

It violates 1NF because a single cell of the hobbies column holds many values at once, so the values are not atomic.

The practical problem: 'find all students who like chess' forces substring searching inside cells instead of a simple WHERE hobby = 'chess'.

Updates are risky too: removing one hobby means editing a packed string, which can corrupt the other hobbies in the same cell.

The column also cannot be indexed or used in a foreign key properly.

Fix: store one hobby per row. Replace one row (1, Aben, 'chess, music') with two rows: (1, Aben, chess) and (1, Aben, music).

Now every cell holds a single value, so the table is in 1NF.

The repeating group is gone, and hobbies become queryable with ordinary SQL.

This is the standard 1NF repair: when one cell holds many values, spread them across rows.

13 Explain 2NF with an example. What is partial dependency? (Model paper, Mar 2021)

REPEATED

Second Normal Form requires: the relation is in 1NF, and it has no partial dependency.

Partial dependency: a non-prime attribute depends on only part of a composite key instead of the whole key.

Worked example: consider Marks(rollno, subject, name, marks) with composite primary key (rollno, subject).

The attribute name depends only on rollno, which is just part of the key, so it is a partial dependency; marks depends on the full key and is fine.

The anomaly: the student name repeats in every subject row, so renaming a student needs many updates, and the name is lost if the last marks row is deleted.

Fix: decompose into Student(rollno, name) and Marks(rollno, subject, marks), keeping rollno as the link.

In Student, name depends on the whole key rollno; in Marks, marks depends on the whole key (rollno, subject).

Both tables are now in 2NF because every non-key attribute depends on the full key of its table.

Exam shortcut: 2NF only matters with a composite key; a single-column key can never have a partial dependency.

14 When does 2NF matter? What if the primary key is a single column?

IMPORTANT

2NF matters only when the primary key is composite, that is, made of two or more columns.

A partial dependency needs 'part of the key' to exist, which is impossible when the key is a single column.

Therefore a relation in 1NF with a single-column primary key is automatically in 2NF.

Example: Student(rollno, name, dept) with key rollno skips 2NF analysis entirely.

It may still fail 3NF: with rollno -> dept and dept -> hod, hod is a transitive dependency, so further decomposition is needed.

So the exam strategy is: check the key first. Composite key means test for partial dependency; single column means move straight to 3NF.

This shortcut saves time and shows the examiner you understand why 2NF exists.

In short: no composite key, no partial dependency, no 2NF problem.

15 **Explain 3NF with an example. What is transitive dependency? (Oct 2019, Mar 2021)**

REPEATED

Third Normal Form requires: the relation is in 2NF, and it has no transitive dependency.

Transitive dependency: X -> Y and Y -> Z, where Z is a non-prime attribute and Y is not a key, so Z depends on X only indirectly through Y.

Worked example: Student(rollno, name, dept, hod) with FDs rollno -> dept and dept -> hod, and rollno as the key.

hod depends transitively on rollno via dept, which violates 3NF.

The anomalies: the HOD name repeats for every student of the department; changing the HOD needs many row updates; deleting the last student of a department loses the HOD fact.

Fix: decompose into Student(rollno, name, dept) and Department(dept, hod).

Department stores each HOD exactly once with dept as key; Student keeps dept as a foreign key.

In each new table every non-key attribute depends directly on the key, so both are in 3NF.

Remember the pattern: non-key depending on non-key means transitive, and transitive means decompose.

16 **A table has rollno -> city and city -> state. Is it in 3NF? Fix it.**

IMPORTANT

No, it is not in 3NF. Take Student(rollno, name, city, state) with rollno as key and FDs rollno -> city and city -> state.

state depends on rollno only indirectly through city, which is a non-key attribute, so this is a transitive dependency.

3NF forbids transitive dependencies, hence the table fails 3NF.

The cost: the state of a city repeats in every student row from that city, causing update anomalies if a state name ever changes.

Fix: decompose into Student(rollno, name, city) and CityState(city, state).

CityState stores each city-state pair exactly once, with city as its primary key.

Student keeps city as a foreign key linking the two tables.

In both tables every non-key attribute depends directly on the key, so both are in 3NF.

17 **Explain various types of normal forms with examples. (Oct 2019, Mar 2021)**

SURE MARK

1NF: no multi-valued or repeating columns; every cell holds one atomic value. Example: split Hobbies('cricket, chess') into separate rows.

2NF: in 1NF, plus no partial dependency - every non-key column must depend on the WHOLE key. Example: Marks(rollno, subject, student_name, mark) has student_name depending only on rollno, so split into Student(rollno, student_name) and Marks(rollno, subject, mark).

3NF: in 2NF, plus no transitive dependency - non-key columns must depend only on the key. Example: Emp(empno, deptno, deptname) has deptname depending on deptno, so split into Emp(empno, deptno) and Dept(deptno, deptname).

BCNF: stricter than 3NF - every determinant must be a candidate key. Example: Teaching(student, subject, teacher), where each teacher teaches one subject, can be in 3NF yet violate BCNF; split into Teaches(teacher, subject) and Enrolled(student, teacher).

Higher forms (4NF, 5NF) handle multi-valued and join dependencies and are rarely asked.

This exact essay repeated verbatim in the Oct 2019 and Mar 2021 Part C - prepare one clean version covering all four.

18 **What is the purpose of normalization in database design? (Model paper)**

IMPORTANT

Normalization organises tables to remove redundancy and the anomalies it causes.

Its purpose is that every fact is stored exactly once, in the right table.

It breaks big messy tables into smaller ones using functional dependencies, from 1NF up to BCNF.

Benefits: no update, insertion or deletion anomalies, less storage waste, and consistent data.

Example: splitting Enroll(rollno, name, course) into Student(rollno, name) and Enroll(rollno, course).

In short, normalization trades a little join effort for a database that stays correct.

19 What is BCNF? How is it stricter than 3NF? (Dec 2021)

IMPORTANT

BCNF (Boyce-Codd Normal Form) rule: for every non-trivial FD X -> Y in the relation, X must be a candidate key.

In plain words: every determinant, every left side of a FD, must be a key.

3NF is looser: it permits a non-key determinant X -> Y provided Y is a prime attribute, that is, part of some candidate key.

BCNF closes that loophole; even when Y is prime, X must still be a key.

Worked example: Teaching(student, subject, teacher) with FDs {student, subject} -> teacher and teacher -> subject.

teacher -> subject passes 3NF because subject is prime, but it violates BCNF because teacher is not a candidate key.

Fix: decompose into TeacherSubject(teacher, subject) and StudentTeacher(student, teacher).

Every relation in BCNF is automatically in 3NF, but the reverse is false; BCNF is the strongest normal form based on FDs.

In the exam, always list the FDs, find the candidate keys, then test each FD's left side: non-key left side means not BCNF.

20 Give an example of a table in 3NF but not in BCNF.

IMPORTANT

Consider Teaching(student, subject, teacher) with FDs {student, subject} -> teacher and teacher -> subject.

Assume each teacher teaches only one subject, but a subject may be taught by several teachers.

Candidate keys: {student, subject} is a key, and {student, teacher} is also a key, because teacher determines subject.

Check 3NF: for teacher -> subject, the right side (subject) is a prime attribute, part of a candidate key, so 3NF permits this FD. The table is in 3NF.

Check BCNF: for teacher -> subject, the left side (teacher) is not a candidate key, and BCNF demands every determinant be a key. So the table is not in BCNF.

The anomaly: a teacher's subject repeats across rows, and deleting a student can destroy the fact that a teacher teaches a subject.

Fix: decompose into TeacherSubject(teacher, subject) and StudentTeacher(student, teacher).

In TeacherSubject, teacher is the key; in StudentTeacher, {student, teacher} is the key.

Both new relations satisfy BCNF, since every determinant is now a key.

21 What is denormalization? When is it used?

IMPORTANT

Denormalization is the deliberate re-introduction of controlled redundancy into a normalized database to improve read performance.

Normalization splits tables to remove duplication; denormalization merges some of that data back, typically to avoid expensive joins.

Example: a reporting table may store the customer name inside every order row, so a sales report needs no join with the Customer table.

It is used mainly in data warehouses and reporting systems, where queries read huge volumes of data but writes are rare and batched.

It is also used on live systems when a specific join-heavy query is a proven bottleneck.

The trade-off: reads get faster, but storage grows and every write must update all copies consistently.

It is therefore applied selectively, only to the hot queries that truly need the speedup.

Denormalization is a performance decision, not a design failure; it is chosen consciously after normalization is understood.

22 What problems can denormalization create?

IMPORTANT

The biggest problem is update anomalies: the same fact stored in two places can drift out of sync when one copy is updated and the other is missed.

Example: if the customer name is copied into every order row, renaming the customer needs updates across thousands of rows, and a partial update leaves contradictory names.

Storage cost rises because data is deliberately duplicated.

Write performance drops: every insert or update must touch all copies, usually inside one transaction.

Consistency guarding moves from the schema to the application, which is error-prone.

Delete anomalies can appear too: removing the last copy of a fact in one table may orphan the copies elsewhere.

These costs are accepted only where read speed matters far more than write speed and storage, such as data warehouses.

So the exam answer is: faster reads, but anomalies, extra storage, slower writes and harder

maintenance.

23 Explain the ACID properties with examples.

IMPORTANT

ACID is the set of four guarantees a DBMS gives every transaction: Atomicity, Consistency, Isolation and Durability.

Atomicity: a transaction is all or nothing. In a fund transfer, the debit and the credit either both happen or both are undone; a crash midway cannot leave money debited but not credited.

Consistency: every transaction moves the database from one valid state to another, preserving all rules. After the transfer, the sum of all balances is unchanged and no account goes negative.

Isolation: concurrent transactions behave as if they ran one after another. Two transfers on the same account do not interleave and corrupt the balance.

Durability: once a transaction commits, its effects survive crashes and power failure. After the transfer commits, the new balances stay even if the server dies a second later.

Together these four properties are why databases are trusted with money, bookings and other critical data.

If any one of them fails, the database can hold wrong or half-written data, which is why the DBMS enforces all four for every transaction.

Memory hook: A for all-or-nothing, C for correct states, I for isolated execution, D for data that survives crashes.

24 What happens if atomicity is violated during a bank transfer?

IMPORTANT

Suppose a transfer debits Rs 5000 from account A, and then the system crashes before crediting account B.

Without atomicity, the debit stays but the credit never happens, so Rs 5000 simply vanishes from the database.

Consistency breaks too: the total money recorded by the bank no longer matches reality.

Customers see wrong balances, statements disagree with the bank's books, and trust collapses.

The DBMS prevents this by treating the debit and credit as one atomic unit: on any failure it rolls back the debit as well.

Rollback restores the pre-transfer state, so the transfer either completes fully or has no effect at all.

That is the point of atomicity: it turns a pair of updates into a safe, all-or-nothing transaction.

Partial work is the most dangerous kind of wrong data: it looks complete but is not.

25 Briefly describe the states of a transaction. (Oct 2019, Mar 2021)

REPEATED

Active: the transaction is running and executing its reads and writes.

Partially committed: the last statement finished; the DBMS now checks whether everything can be made permanent.

Committed: all changes are safely recorded - the transaction succeeded.

Failed: something went wrong (error or crash) and normal execution cannot continue.

Aborted: after failure, the transaction is rolled back and the database is restored to its earlier state.

A transaction moves Active -> Partially committed -> Committed on success, or Active/Partially committed -> Failed -> Aborted on failure.

26 What is a transaction? Explain three problems of concurrent execution.

IMPORTANT

A transaction is a logical unit of work: a sequence of database operations that must either complete fully or have no effect at all. Example: a fund transfer is a debit plus a credit as one transaction.

Problem 1, lost update: T1 and T2 both read balance 1000, each adds 100, and each writes back its result. The final balance is 1100 instead of 1200 because the second write overwrites the first; one update is lost.

Problem 2, dirty read: T1 reads a balance that T2 has written but not yet committed. If T2 then rolls back, T1 has acted on data that never officially existed.

Problem 3, unrepeatable read: T1 reads a row, T2 changes and commits it, then T1 reads the same row again and gets a different value, so T1 cannot trust its own earlier read.

All three arise only when transactions run concurrently without control.

They are exactly what concurrency control prevents: locks, two-phase locking and serializability rules exist to rule out these failures.

In the exam, always give the T1/T2 story for each problem; the concrete example is what earns full marks.

Remember the trio as: lost update (overwrite), dirty read (uncommitted), unrepeatable read (changed between reads).

27 What is a schedule? What makes a schedule correct?

IMPORTANT

A schedule is the chronological order in which the operations of concurrently running transactions are interleaved by the DBMS.

Example: a schedule of T1 and T2 could be read1(A), read2(A), write1(A), write2(A), commit1, commit2.

A schedule is correct mainly if it is serializable: its final effect equals the effect of running the same transactions one after another in some serial order.

It should also be recoverable: no transaction commits based on uncommitted data of another, so crash recovery stays possible.

The DBMS scheduler enforces these properties using locking protocols like 2PL or timestamp ordering.

In short: a schedule is correct when concurrency changed the execution order but not the outcome.

Serial schedules (no interleaving at all) are always correct but slow; serializable schedules give the same correctness with far better performance.

That trade-off is the whole reason concurrency control theory exists.

28 Explain shared and exclusive locks with an example.

IMPORTANT

Locks are how the DBMS stops two transactions from changing the same data at the same time; a transaction must hold a lock on a data item before touching it.

Shared lock (S): taken for reading. Many transactions can hold shared locks on the same item together, because reading does not change anything.

Exclusive lock (X): taken for writing. Only one transaction can hold it, and while it is held nobody else may read or write the item.

Compatibility rule: S is compatible with S, but X conflicts with both S and X, so a writer waits for all readers and other writers.

Example: T1 takes a shared lock on the balance row to read it; T2 wants to update the balance and requests an exclusive lock, so T2 waits until T1 releases.

Rule of thumb: take a shared lock before reading, an exclusive lock before writing, and release locks when the transaction finishes.

This discipline is what prevents lost updates and dirty reads.

Without locks, concurrent transactions would freely overwrite each other's work.

29 What happens if a transaction reads data without taking any lock?

IMPORTANT

It can suffer a dirty read: it may see a value written by another transaction that later rolls back, so the data it acted on never officially existed.

It can suffer an unrepeatable read: reading the same row twice may return different values, because a concurrent transaction changed it in between.

It can read a half-updated state: with no lock, nothing stops a writer from modifying the row in the middle of the read.

Its own later writes can then be based on stale values, producing lost updates that corrupt totals.

Locks exist precisely to rule out these failures: every read takes a shared lock and every write takes an exclusive lock.

Reading without a lock trades a little speed for completely unreliable results.

A DBMS must never allow that trade-off, because wrong data silently spreads into every later computation.

So the rule is absolute: no lock, no touch.

30 What affects the performance of locking?

IMPORTANT

Lock granularity: row-level locks let many transactions work on different rows of the same table at once, while table-level locks block the whole table and destroy concurrency.

Lock overhead: acquiring, checking and releasing locks costs CPU time and memory, so heavy locking slows even simple operations.

Blocking and waiting: a transaction stuck waiting for a lock held by another delays its whole chain, and long waits cascade into deadlocks.

Lock duration: holding locks until commit, as in strict 2PL, is safest but blocks others longest; releasing early gains speed at the cost of safety.

Lock escalation: when a transaction holds too many fine locks, the system may merge them into one coarse lock, suddenly reducing concurrency.

Deadlock handling itself costs performance: detection scans the wait-for graph, and victim rollback wastes completed work.

Good design balances the two sides: locks fine-grained and short-lived enough for speed, strict enough to keep data correct.

In the exam, granularity, overhead and waiting time are the three points that must appear.

31 What is Two-Phase Locking? Explain its two phases.

IMPORTANT

Two-Phase Locking (2PL) is a protocol that guarantees serializable schedules by splitting a transaction's locking behaviour into two strict phases.

Phase 1, growing: the transaction may acquire locks but may not release any lock.

Phase 2, shrinking: the transaction may release locks but may not acquire any new lock.

The lock point, the moment the first lock is released, separates the two phases; after it, no new locks are allowed.

Example: T1 locks A, locks B, updates both, then releases A and B. All lock requests happened before any release, so 2PL holds.

Counter-example: T1 locks A, releases A, then locks B. This violates 2PL because a lock is acquired after a release.

2PL guarantees conflict serializability: any schedule of 2PL transactions has the same effect as some serial order.

Limitation: plain 2PL does not prevent cascading rollbacks. Strict 2PL, which holds all exclusive locks until commit, fixes this and is what real systems use.

So 2PL gives correctness, and strict 2PL adds recoverability on top.

32 What is serializability? Why does it matter?

IMPORTANT

A schedule is serializable if its final effect on the database is identical to the effect of running the same transactions one after another in some serial order.

Example: if an interleaved schedule of T1 and T2 leaves the same balances as running T1 fully and then T2 fully, the schedule is serializable.

It matters because it is the correctness standard for concurrency: it proves that interleaving operations did not corrupt anything.

Without serializability, lost updates, dirty reads and inconsistent totals can slip through silently.

2PL is the standard practical way to guarantee serializability.

Serial schedules are trivially serializable but slow; serializable schedules give the same safety with far better performance.

Conflict serializability, tested with a precedence graph, is the version the DBMS actually checks.

In one line: serializability means concurrency changed the order but not the outcome.

33 What is a recoverable schedule?

IMPORTANT

A schedule is recoverable if no transaction commits before the transactions whose uncommitted data it read have committed.

Example: if T1 reads a value written by T2, then T1 must commit only after T2 commits; letting T1 commit first makes the schedule unrecoverable.

Why it matters: if T2 rolls back after T1 has already committed, the DBMS cannot undo T1's committed work, leaving the database permanently inconsistent.

A stronger form is cascadelessness: a transaction reads only committed data, so one rollback never forces a chain of other rollbacks.

Strict 2PL, which holds all exclusive locks until commit, automatically produces cascadeless and therefore recoverable schedules.

Recoverability is what makes crash recovery possible: the log can safely redo and undo because the commit order respected read dependencies.

Without it, a crash could leave committed transactions depending on rolled-back data, which no recovery procedure can repair.

So recoverability is the rule that keeps the commit order honest.

34 Differentiate between conflict serializability and view serializability. (Model paper)

IMPORTANT

Both test whether a concurrent schedule behaves like some serial (one-after-another) order.

Conflict serializability: a schedule is safe if its conflicting operations (read-write or write-write on the same item) can be reordered into a serial order; tested with a precedence graph - no cycles means safe.

View serializability: a weaker test - the schedule only needs to give each read the same value, and the same final writes, as some serial schedule.

Every conflict-serializable schedule is view-serializable, but not vice versa.

In practice DBMSs enforce conflict serializability (via 2PL) because it is easy to test; view serializability is harder to check.

Remember the one-line difference: conflict looks at operation order, view looks at what each read actually saw.

35 What is a deadlock? How is it handled in DBMS?

IMPORTANT

A deadlock is a circular wait between transactions: each transaction holds a lock that another one needs, so none of them can ever proceed.

Example: T1 locks row A and requests row B; T2 locks row B and requests row A. Each waits for the other forever.

Handling method 1, prevention: impose a global order on lock requests, always lock A before B, so waiting cycles can never form.

Handling method 2, detection and recovery: the DBMS builds a wait-for graph and checks it periodically; on finding a cycle it picks a victim transaction and rolls it back, freeing its locks.

Handling method 3, timeouts: if a transaction waits longer than a fixed limit, the system aborts it and retries, breaking the wait cheaply.

Prevention is simple but restrictive; timeouts are simple but may abort transactions that were not really deadlocked.

Most real DBMSs use detection with victim selection, since it allows maximum concurrency and only pays the cost when a deadlock truly occurs.

The victim is usually the transaction with the least work done or the lowest priority, so the wasted work is minimal.

36 How does the wait-for graph detect a deadlock?

IMPORTANT

The wait-for graph has one node for each active transaction in the system.

A directed edge T1 -> T2 is drawn whenever T1 is waiting for a lock currently held by T2.

The DBMS updates this graph as transactions request and release locks, and checks it periodically for cycles.

A cycle means a deadlock: following the edges loops back to the start, so every transaction in the cycle waits on another one in the cycle and nobody can proceed.

Example: edges T1 -> T2 and T2 -> T1 form a cycle, proving T1 and T2 are deadlocked over the locks they hold.

Recovery: the system chooses a victim, usually the transaction with the least completed work or lowest priority, rolls it back and releases its locks, which breaks the cycle.

The surviving transactions then continue normally, and the victim is typically retried.

The graph turns an invisible waiting problem into a visible cycle that the DBMS can detect and fix automatically.

37 Explain COMMIT, ROLLBACK and SAVEPOINT with examples. (Model paper)

IMPORTANT

COMMIT ends a transaction successfully and makes all its changes permanent and durable.

ROLLBACK ends a transaction by undoing every change made since the last commit, restoring the old state.

SAVEPOINT marks a named point inside a transaction, so a later rollback can stop partway instead of undoing everything.

Example of all three together:

```
UPDATE Account SET balance = balance - 500 WHERE id = 1;
SAVEPOINT s1;
UPDATE Account SET balance = balance + 500 WHERE id = 2;
ROLLBACK TO s1;
COMMIT;
```

Here the debit of account 1 stays, but the credit to account 2 is undone by rolling back to s1, and COMMIT then saves the debit permanently.

Savepoints are useful when one step of a multi-step transaction fails but the earlier steps are still valid and worth keeping.

38 How does a DBMS recover from a crash?

IMPORTANT

The DBMS keeps a write-ahead log: every change is recorded in the log on disk before it is applied to the database itself.

When the system restarts after a crash, the recovery manager scans this log from the last checkpoint.

Committed transactions are redone: their logged changes are reapplied so their effects survive the crash, which gives durability.

Uncommitted transactions are undone: their partial changes are rolled back so no half-done work remains, which gives atomicity.

Checkpoints, periodic snapshots of the database state written to the log, shorten the scan by marking how far back recovery must look.

Because of the write-ahead rule, a crash can never leave the database in a half-updated state.

Everything is either fully applied or fully removed, so the database always restarts in a consistent state.

This is why the log is the most critical structure in a DBMS: it is the difference between a crash

and a catastrophe.

39 Explain the types of failures in a DBMS. (Dec 2021)

IMPORTANT

Transaction failure: a single transaction aborts due to an error (bad input, deadlock); only its changes are rolled back.

System crash: power loss or OS failure wipes memory; committed data on disk survives, uncommitted work is undone using the log.

Disk failure: the storage itself is damaged; recovery needs backups plus the log replayed from the last backup.

The DBMS handles the first two with write-ahead logging (undo/redo); disk failure needs archived backups.

Knowing the failure types decides the recovery strategy, which is why the exam asks it before crash recovery.

Failure types decide the recovery method, so this question pairs naturally with crash recovery in the same exam.

MODULE 4

NoSQL Databases and Big Data

4

EXAM PATTERN (from real MGU question papers)

MGU BCA Honours DBMS (MG3CCRBCA201): 70 marks, 2 hours. Part A: 5 x 2 marks, short questions from all modules. Part B: answer any 5 of 6-mark questions. Part C: answer any 2 of 15-mark essays. Old CBCS papers (2019-2021): 80 marks, 3 hours, any-10 x 2 + any-6 x 5 + any-2 x 15. Part C hotspots: COMPANY ER diagram, Normal Forms essay, anomalies.

1 What is NoSQL? Why was it introduced?

IMPORTANT

NoSQL stands for Not Only SQL. It is a family of database systems built to handle very large, fast-changing and unstructured data that traditional relational databases struggle with.

It was introduced because web and mobile apps like social media, e-commerce and streaming started generating data in volumes and forms (posts, videos, logs, clicks) that a single-server SQL database could not store or process fast enough.

NoSQL databases scale horizontally, which means they add more cheap servers to share the load instead of buying one bigger server. This is called scaling out.

They use flexible schemas, so documents or records can have different fields without redesigning tables first.

To gain speed and availability they usually trade off the strict ACID consistency of SQL for a looser model called BASE.

Common examples are MongoDB (documents), Cassandra (column families), Redis (key-value) and Neo4j (graph).

2 Give two differences between SQL and NoSQL databases.

IMPORTANT

SQL and NoSQL databases differ in four main ways: data organisation, schema, scaling and consistency.

First, data organisation: SQL stores data in fixed-schema tables of rows and columns, with relationships handled through joins, while NoSQL uses flexible models like documents, key-value pairs, column families or graphs.

Second, schema: in SQL the schema must be defined upfront and changing it later with ALTER TABLE is costly, whereas NoSQL is schema-free so each record can carry its own fields and the structure evolves freely.

Third, scaling: SQL scales vertically by moving to a bigger single server, which is expensive and has hard limits, while NoSQL scales horizontally by adding cheap servers, which handles

massive data and traffic.

Fourth, consistency: SQL guarantees ACID, so every transaction leaves the data fully correct at once, while NoSQL follows BASE and is eventually consistent, trading instant correctness for speed and availability.

Fifth, query style: SQL uses the standard SQL language everywhere, while each NoSQL system has its own API suited to its data model.

Example: use SQL for bank accounts where exact balances matter, and use NoSQL for a social media feed where speed and scale matter more than instant consistency.

In short, choose SQL when correctness is non-negotiable and NoSQL when huge scale, flexibility and uptime come first.

3 Explain the four NoSQL data models with one example each. (Model paper)

IMPORTANT

NoSQL systems organise data in four main models, and each model fits a different kind of problem.

1. Document model: data is stored as flexible JSON-like documents, each with its own fields, and related data can be nested inside one document. Example: a product document {name: 'shoe', price: 999, reviews: [...]}. Used by MongoDB for catalogs, profiles and content management.

2. Key-value model: every key maps to exactly one value, like a giant dictionary, giving extremely fast lookups by key. Example: key 'session:1234' maps to a user's cart data. Used by Redis for caching, sessions and shopping carts.

3. Column-family model: data is stored in rows made of grouped columns called column families, which is efficient for huge sparse tables where most cells are empty. Example: a user row with column families profile: and activity:. Used by Cassandra for time-series and sensor data.

4. Graph model: data is stored as nodes and edges, so relationships are first-class citizens and traversals are fast. Example: node 'Aben' -FRIENDS_WITH-> node 'Ashik'. Used by Neo4j for social networks and fraud detection.

The rule for choosing: match the model to the shape of your data and the queries you will run most often.

4 When would you choose a graph database over a document database?

IMPORTANT

Choose a graph database over a document database when the relationships between items matter more than the items themselves.

In a graph database, connections are stored directly as edges, so queries like 'friends of friends' or 'shortest path' are single fast traversals. Examples are social networks, fraud detection tracing suspicious money paths, and recommendation engines.

In a document database, each record is self-contained and flexible, which is ideal when you mostly fetch whole records by key. Examples are product catalogs, user profiles and blog posts, where each record has different fields and nested data.

The practical difference: finding friends-of-friends in a graph is one traversal, while in a document store it needs multiple lookups or application-level joins, which gets slow as connections grow.

So the rule of thumb is: graphs for connected data and traversal queries, documents for independent flexible records.

5 What are the features and uses of NoSQL document databases?

IMPORTANT

Document databases store data as flexible JSON-style documents grouped into collections, much like rows grouped into tables in SQL.

Flexible schema: documents in one collection can have different fields, so you can add a new field to new documents without rebuilding anything.

Nested data: related details can live inside one document, for example a product document that contains its reviews. This keeps related data together and removes the need for many joins.

Horizontal scaling: document databases spread data across many servers, so they handle large applications and growing traffic well.

Developer friendly: JSON matches the data formats used in web and mobile apps, so moving data between the app and the database is easy.

Uses: product catalogs, user profiles, content management systems, blogging platforms and real-time analytics, anywhere records vary in shape and change often.

6 Why are document databases a good fit for an online shopping site?

IMPORTANT

Document databases fit online shopping sites because a store's data is varied, nested and constantly changing.

Products differ in attributes: a T-shirt has sizes and colours while a book has an author and page count. A document store lets each product keep its own fields with no rigid table schema forcing empty columns.

Nested documents keep everything together: a product document can contain its reviews, images and stock details inside it, so one read loads the whole product page fast.

Schema changes are painless: adding a new field like 'discount' to new products needs no table rebuild and does not disturb the millions of existing documents.

Scaling out handles traffic: during festival sales, more servers are added to carry the load, which document stores like MongoDB are built for.

JSON documents also match web and mobile apps naturally, since APIs already speak JSON, so less conversion code is needed.

In short, a document database matches exactly how online stores evolve: varied products, fast page reads and constant change.

7 Explain the CAP theorem.

IMPORTANT

The CAP theorem states that a distributed database system can fully guarantee only two of three properties at the same time: Consistency, Availability and Partition tolerance.

C, Consistency: every read receives the most recent write. All nodes see the same data at the same time.

A, Availability: every request gets a response, even when some servers have failed. The system never refuses to answer.

P, Partition tolerance: the system keeps working even when the network splits and servers cannot talk to each other. Partitions do happen in real networks.

Since network failures are unavoidable, partition tolerance cannot really be given up, so designers choose between the remaining two.

This gives two practical types: CP systems like HBase and MongoDB (when configured strictly) pick consistency over availability, and AP systems like Cassandra pick availability over strict consistency.

Example: banks usually pick CP so balances are always exact, while social feeds pick AP so

the feed always loads even if some updates arrive late.

8 In the CAP theorem, why must partition tolerance be kept?

IMPORTANT

In the CAP theorem, partition tolerance must be kept because network partitions, where some servers cannot reach others, are unavoidable in any real distributed system.

Cables fail, routers crash, switches misbehave and whole cloud regions get isolated. No network is perfect, and designing as if partitions never happen is unrealistic.

Giving up partition tolerance would mean the entire system stops working the moment any network split occurs, which is unacceptable for a service that must stay up.

Therefore designers accept P as mandatory and make the real choice between consistency and availability.

That choice is guided by the application: banking systems keep consistency (CP) because showing a wrong balance is costly, while social feeds keep availability (AP) because showing a slightly stale post is harmless.

So partition tolerance is not a preference but a fact of distributed life, and the C-versus-A trade-off is where the real design decision lies.

9 Distinguish between ACID and BASE.

IMPORTANT

ACID and BASE are two different philosophies for keeping data correct, used by SQL and NoSQL databases respectively.

ACID stands for Atomicity, Consistency, Isolation and Durability. Atomicity means a transaction is all or nothing, never half done. Consistency means every transaction takes the database from one valid state to another. Isolation means concurrent transactions do not interfere with each other. Durability means once committed, data survives crashes.

BASE stands for Basically Available, Soft state and Eventually consistent. Basically Available means the system keeps responding even during failures. Soft state means data may change over time without new input while copies converge. Eventually consistent means all copies will agree after some delay.

Point by point: ACID demands instant consistency while BASE accepts temporary differences; ACID may refuse a write to stay correct while BASE accepts the write to stay available; ACID suits single strong servers while BASE suits huge distributed clusters.

Use ACID where exact correctness is non-negotiable, like banking and ticket booking, and BASE where massive scale and uptime matter more, like feeds, carts and catalogs.

10 What does "eventually consistent" mean? Give an example.

IMPORTANT

Eventually consistent means that after a write, all copies of the data will agree with each other after some delay, but not instantly.

In a distributed system the same data lives on many servers. A write reaches one server first and then spreads to the others through replication.

During that short window, different users can see different values for the same item. This temporary disagreement is the 'soft state' in BASE.

Example: you like a post on your phone and the count shows 101 immediately, while your friend on another server still sees 100 for a few seconds, and then both settle at 101.

This trade-off is deliberate: it keeps huge systems fast and available instead of making every user wait for every copy to update.

It is acceptable for feeds, likes and catalogs where a few seconds of staleness harms nobody, but not for banking where balances must be exact.

11 Explain CRUD operations in MongoDB with examples.

IMPORTANT

CRUD stands for Create, Read, Update and Delete, the four basic operations every database supports. In MongoDB each is done with a simple command on a collection.

Create: add a new document to a collection. Example: db.students.insertOne({name: 'Aben', marks: 85}) creates one student document.

Read: fetch documents that match a condition. Example: db.students.find({marks: {$gt: 80}}) returns all students whose marks are greater than 80, where $gt is the greater-than operator.

Update: change fields of matching documents. Example: db.students.updateOne({name: 'Aben'}, {$set: {marks: 90}}) sets Aben's marks to 90, where $set is the set-field operator.

Delete: remove matching documents. Example: db.students.deleteOne({name: 'Aben'}) removes Aben's document.

Other useful operators: $lt for less than, $eq for equal, $in to match any value in a list, and $gte/$lte for greater/less than or equal. Together CRUD and these operators cover nearly all everyday database work.

12 Write a MongoDB query to find students with marks between 60 and 90.

IMPORTANT

To find students with marks between 60 and 90, combine the $gte (greater than or equal) and $lte (less than or equal) operators in one find query.

```
db.students.find({marks: {$gte: 60, $lte: 90}});
```

How it works: find() searches the students collection and returns every document matching the filter.

$gte: 60 keeps only documents with marks 60 or above, and $lte: 90 keeps only documents with marks 90 or below, so together they form the 60-to-90 range.

Both conditions apply to the same marks field inside one document filter, which is why they sit together in {marks: {...}}.

Example result: if Aben has 85 and Ashik has 55, only Aben's document is returned.

Change the two numbers to get any range, and add .pretty() at the end for readable output.

13 Write about update operators in MongoDB. (Model paper)

IMPORTANT

Update operators modify fields inside MongoDB documents without rewriting the whole document.

$set changes a field's value:

```
db.students.updateOne({rollno: 101}, {$set: {marks: 95}});
```

$inc increments a number:

```
db.students.updateOne({rollno: 101}, {$inc: {marks: 5}});
```

$unset removes a field; $push appends a value to an array field.

Update operators are used with updateOne, updateMany or replaceOne, and they make MongoDB updates precise and fast.

14 Write short notes on Hadoop, MongoDB and Cassandra.

IMPORTANT

Hadoop, MongoDB and Cassandra are three pillars of big data, each solving a different part of the problem.

Hadoop is an open-source platform for storing and processing huge data across clusters of ordinary machines. HDFS (Hadoop Distributed File System) splits files into blocks spread over many machines with replication for safety, and MapReduce processes the blocks in parallel. Example: analysing one year of web server logs.

MongoDB is a schema-free document database that stores big varied data as flexible JSON-like documents. Example: storing product catalogs or user profiles where every record has different fields.

Cassandra is a distributed column-family database built for high availability and massive write loads, with no single point of failure. Example: storing sensor readings or chat message inboxes arriving thousands per second.

Together they cover the three big-data needs: Hadoop for batch storage and analysis, MongoDB for flexible storage, and Cassandra for always-on heavy writes.

15 A company gets one crore sensor readings a day. Which technology fits and why?

IMPORTANT

For one crore sensor readings a day, Hadoop is the best fit for the storage and analysis part.

HDFS stores the huge volume cheaply across many ordinary machines, and its replication keeps the data safe even when machines fail.

MapReduce processes all the readings in parallel, so daily reports like average temperature per hour finish quickly instead of taking all day on one machine.

If the company also needs instant lookups, Cassandra fits the live-serving part: it handles massive write loads and stays available, which suits dashboards and alerts on fresh readings.

MongoDB is less ideal here because its strength is varied flexible documents, while sensor readings are uniform time-series data.

So the practical answer is: Hadoop for batch storage and daily analysis of the one crore readings, plus Cassandra alongside it for the live serving layer.

16 Define Big Data. (Model paper)

IMPORTANT

Big Data means datasets so large, fast or varied that traditional databases cannot handle them.

It is described by the 3 Vs: Volume (terabytes to petabytes), Velocity (streaming sensor and click data), Variety (text, images, video, logs).

Example: one crore sensor readings a day, or every click on a shopping website.

Such data needs distributed systems like Hadoop and NoSQL stores like MongoDB or Cassandra.

Big Data problems appear when volume, velocity or variety breaks a single relational server.

That is why the syllabus pairs it with Hadoop, MongoDB and Cassandra - the distributed answer.

MODULE 5

Practical

5

EXAM PATTERN (from real MGU question papers)

MGU BCA Honours DBMS (MG3CCRBCA201): 70 marks, 2 hours. Part A: 5 x 2 marks, short questions from all modules. Part B: answer any 5 of 6-mark questions. Part C: answer any 2 of 15-mark essays. Old CBCS papers (2019-2021): 80 marks, 3 hours, any-10 x 2 + any-6 x 5 + any-2 x 15. Part C hotspots: COMPANY ER diagram, Normal Forms essay, anomalies.

1 Draw an ER diagram for a college database with Student, Course and Teacher. Describe it.

IMPORTANT

This practical task is to draw an ER diagram for a college database with three entities: Student, Course and Teacher.

Entities with their attributes (key attributes are underlined in the diagram):

```
Student(rollno, name) where rollno is the primary key.
Course(code, title) where code is the primary key.
Teacher(id, name) where id is the primary key.
```

Relationships: Student enrolls in Course (many-to-many); Teacher teaches Course (one-to-many).

Draw rectangles for the three entities Student, Course and Teacher.

Draw diamonds for the relationships Enrolls and Teaches, and connect each diamond to its two entity rectangles with lines.

Draw ellipses for attributes and connect them to the entity rectangles; underline the key attribute ellipse of each entity.

Many-to-many Enrolls would become its own table with rollno and code as foreign keys, while the one-to-many Teaches is handled by a foreign key of Teacher in the Course table.

2 How do you convert a many-to-many relationship into tables?

IMPORTANT

A many-to-many relationship cannot be stored in either entity table directly, so it is converted into its own table.

Create a separate table for the relationship itself.

```
CREATE TABLE Enrolls (
rollno INT,
code VARCHAR(10),
grade CHAR(2),
```

```
PRIMARY KEY (rollno, code),
FOREIGN KEY (rollno) REFERENCES Student(rollno),
FOREIGN KEY (code) REFERENCES Course(code)
);
```

Example: Enrolls(rollno, code) with both columns as foreign keys.

The pair (rollno, code) together forms the composite primary key, so one student cannot be enrolled in the same course twice.

Extra attributes of the relationship, like grade, go in this table too.

This new table is called a junction table; it removes the many-to-many link and replaces it with two one-to-many links.

3 Write SQL to create an Employee table and insert three rows.

IMPORTANT

This practical task is to create an Employee table with constraints using DDL and then add three rows using DML.

Create the table:

```
CREATE TABLE Employee (
id INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
salary INT CHECK (salary > 0)
);
```

Insert rows:

```
INSERT INTO Employee VALUES (1, 'Anu', 30000);
INSERT INTO Employee VALUES (2, 'Ben', 45000);
INSERT INTO Employee VALUES (3, 'Cia', 38000);
```

The PRIMARY KEY on id guarantees every employee row has a unique id.

NOT NULL on name stops blank names from being inserted, and CHECK (salary > 0) rejects zero or negative salaries.

Each INSERT adds one complete row; after the three statements the table holds 3 employees.

Expected output: the table Employee with columns id, name, salary and three rows for Anu, Ben and Cia.

4 **Write queries to give Ben a 5000 raise and to delete the employee named Cia. (Mar 2021)**

IMPORTANT

This practical task is to modify an existing Employee table: raise the salary of Ben and remove the row of Cia.

Raise salary with UPDATE and WHERE:

```
UPDATE Employee SET salary = salary + 5000 WHERE name = 'Ben';
```

Remove the row with DELETE and WHERE:

```
DELETE FROM Employee WHERE name = 'Cia';
```

The WHERE clause in the UPDATE limits the change to only the row where name is Ben, so other salaries stay untouched.

Forgetting WHERE in UPDATE or DELETE would change or remove every row, so always double-check the condition first.

Expected result: Ben's salary becomes 50000 and the table has only Anu and Ben left after Cia's row is deleted.

5 **Explain UNION, INTERSECT and EXCEPT with one example each. (Dec 2021, Mar 2021)**

REPEATED

Set operators UNION, INTERSECT and EXCEPT combine the results of two queries into one result.

UNION: students from section A plus section B, duplicates removed.

```
SELECT name FROM SecA UNION SELECT name FROM SecB;
```

INTERSECT: names present in both sections.

```
SELECT name FROM SecA INTERSECT SELECT name FROM SecB;
```

EXCEPT: names in A but not in B.

```
SELECT name FROM SecA EXCEPT SELECT name FROM SecB;
```

Both queries must return the same number and type of columns, otherwise the operator gives an error.

UNION removes duplicate rows; UNION ALL keeps the duplicates.

Expected output: each query returns a single name column with the combined, common, or differing names of the two sections.

6 Write a query to list student names with their subject and mark using INNER JOIN.

IMPORTANT

This practical task is to combine data from two tables, Student and Marks, so that each student appears with his subject and mark.

Join on the common rollno column:

```
SELECT s.name, m.subject, m.mark
FROM Student s INNER JOIN Marks m
ON s.rollno = m.rollno;
```

INNER JOIN matches rows where the ON condition is true, so only students having marks rows appear in the result.

The aliases s and m are shortcuts for the Student and Marks tables, making the query shorter and easier to read.

Expected output: one row per student-mark pair showing name, subject and mark; a student with no marks row is left out.

7 Write a query to find departments with average marks above 70, sorted high to low.

IMPORTANT

This practical task is to find departments whose average marks are above 70 and list them from the highest average to the lowest.

Group by department, filter groups with HAVING, sort with ORDER BY:

```
SELECT dept, AVG(marks) AS avg_marks
FROM Student GROUP BY dept
HAVING AVG(marks) > 70
ORDER BY avg_marks DESC;
```

GROUP BY dept collects the students of each department into groups, and AVG(marks) computes the average of each group.

HAVING filters groups after grouping (WHERE filters single rows before grouping), keeping only departments with average above 70.

ORDER BY avg_marks DESC sorts the surviving departments from the highest average to the lowest.

Expected output: one row per qualifying department with its dept name and avg_marks, highest first.

8 What is PL/SQL? Show the structure of a PL/SQL block with an example.

IMPORTANT

PL/SQL is Oracle procedural language that adds variables and logic to SQL.

It is used for triggers, procedures and batch jobs inside the database.

Basic block structure:

```
DECLARE
m NUMBER;
BEGIN
SELECT marks INTO m FROM Student WHERE rollno = 1;
DBMS_OUTPUT.PUT_LINE('Marks: ' || m);
EXCEPTION
WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE('No student found');
END;
/
```

DECLARE holds variables; BEGIN holds the logic; EXCEPTION handles errors.

SELECT ... INTO m reads a value from the table into the variable m.

The EXCEPTION part catches the NO_DATA_FOUND error when no student has rollno 1 and prints a friendly message instead of crashing.

9 Write a PL/SQL block that prints "Pass" if marks are above 40, else "Fail".

IMPORTANT

This practical task is to write a PL/SQL block that prints "Pass" if marks are above 40, else prints "Fail".

Use an IF condition inside the BEGIN block:

```
DECLARE
m NUMBER := 55;
BEGIN
IF m > 40 THEN
DBMS_OUTPUT.PUT_LINE('Pass');
ELSE
DBMS_OUTPUT.PUT_LINE('Fail');
END IF;
END;
/
```

The variable m is given the value 55 in the DECLARE section, so the condition m > 40 is true.

IF ... THEN ... ELSE ... END IF is the PL/SQL way of choosing between two actions.

DBMS_OUTPUT.PUT_LINE prints the text to the output window.

Expected output: Pass, because 55 is above 40; with m = 30 the output would be Fail.

10 Write MongoDB commands to create a students collection, insert two documents and find toppers.

IMPORTANT

This practical task is to build a small students database in MongoDB: create a collection, insert two documents, and find the toppers.

Switch database and create the collection:

```
use college;
db.createCollection("students");
```

Insert documents:

```
db.students.insertMany([
{name: "Aben", marks: 92},
{name: "Ben", marks: 74}
]);
```

Find students scoring above 85:

```
db.students.find({marks: {$gt: 85}});
```

use college switches to the college database and creates it if it does not exist.

insertMany adds both student documents in one call; MongoDB collections are flexible, so no fixed schema is needed.

The $gt operator means greater than, so find returns only documents with marks above 85.

Expected output: one document for Aben with marks 92; Ben with marks 74 is left out of the result.

11 Write MongoDB commands to update marks and delete a student.

IMPORTANT

This practical task is to update several documents at once and then delete one specific document in MongoDB.

Update matching documents:

```
db.students.updateMany({marks: {$lt: 40}}, {$set: {result: "fail"}});
```

Delete one document:

```
db.students.deleteOne({name: "Ben"});
```

updateMany finds every document whose marks are below 40 ($lt means less than) and adds a result field with value "fail" using $set.

The $set operator changes only the listed fields and leaves the rest of the document untouched.

deleteOne removes the first document matching the filter, here the student named Ben.

Expected result: all students with marks below 40 get result: "fail", and Ben's document is gone from the collection.